

Today's Work: The CNN Module and Two New Bugs

Closing out Module 2, introducing Module 3, and two more real debugging lessons earned from your own code.

Closing out Module 2

Subject-wise normalization — z-scoring each patient's features against their own mean and standard deviation rather than a single global one — was implemented and, once actually running correctly, moved mean LOSO accuracy from 0.48 (a fully degenerate, majority-guessing model) to 0.64, with roughly a third of patients showing genuine, mixed, well-discriminated predictions. Two honest limitations were noted rather than papered over: patients with very few seizures give an unstable per-patient baseline to normalize against, and restricting every patient to one shared 18-channel set may discard exactly the channel that mattered most for some individual patients. This is a legitimate, well-characterized place to call the classical baseline complete.

Module 3: CNNs — Learning Features Instead of Hand-Crafting Them

Module 2 relied on features a human chose in advance: band power, line length. Module 3 removes that ceiling by letting a Convolutional Neural Network learn its own filters directly from the raw signal — patterns nobody explicitly specified.

What a 1D convolution actually does

A convolutional filter is a small, learnable window that slides along the time axis of a signal, computing a simple weighted sum at each position. Where the local shape of the signal matches whatever pattern the filter has learned to respond to, its output is large; elsewhere, small. Stacking many such filters means the network can end up with different detectors for different shapes — spikes, slow oscillations, and combinations no one specified by hand.

The training cycle, made explicit

Rather than a single black-box 'train' call, the actual mechanics were kept visible: a **forward pass** (compute a prediction), a **loss** (a single number measuring how wrong that prediction was), a **backward pass** (compute how much each individual weight contributed to that error, via backpropagation), and an **optimizer step** (nudge every weight slightly toward less error). Repeated many times, this is what shapes random initial filters into useful pattern detectors.

A real design consequence: fixed-length windows

Module 2's features could be computed from an epoch of any length — a seizure lasting 30 seconds and one lasting 90 seconds both reduced to the same 108 numbers. A CNN has no such

flexibility: every example in a batch tensor must be exactly the same shape. This forced a real change from Module 2 — every epoch, seizure or not, is now cut to precisely the same fixed duration.

Two New Debugging Principles

Continuing directly from the earlier debugging field guide — same habit of mind, two new real examples.

Principle 10: When you slice a smaller piece out of something, make sure later code checks the piece — not the whole thing

A variable holding an entire multi-hour recording and a variable holding a small slice cut from it can easily end up with very similar-sounding names. A check written against the wrong one of the two won't error — it will just silently evaluate a condition that was never the one intended.

Case from your own work — the 'data vs. epoch' bug

A safety check meant to confirm a small extracted window was exactly the right length was accidentally written to check the length of the entire original, multi-hour recording instead. Since a full recording is essentially never exactly the target window length, this condition was true almost 100% of the time — silently discarding nearly every real example in the dataset, with no error at any point.

The transferable lesson: When two variables represent 'the whole thing' and 'a piece of the whole thing,' double-check every line that follows the slice to confirm it's still referring to the piece — especially after retyping or hand-editing code rather than copying it exactly.

Principle 11: A single-digit typo in a default parameter can silently gut a result without ever crashing

Default function arguments are easy to mistype quickly, and a wrong number there produces no error at all — the function runs, returns a value, and looks completely normal from the outside.

Case from your own work — the 1–4 Hz filter

A band-pass filter meant to keep the 1–40 Hz range (where seizure-related activity lives) was typed as 1–4 Hz instead — one missing zero. The code ran without any error, but it was silently discarding nearly the entire useful EEG spectrum before the model ever saw it.

The transferable lesson: When a numeric default parameter controls something scientifically meaningful (a frequency range, a threshold, a window size), read it back out loud against its intended value after typing it — typos here are invisible to the program and only show up as unexplained bad results downstream.

Both bugs share the same shape: nothing crashed, everything looked like it ran successfully, and the only way either was caught was by refusing to accept a suspicious result at face value — the exact same instinct behind Principle 3 and Principle 4 in the original field guide.